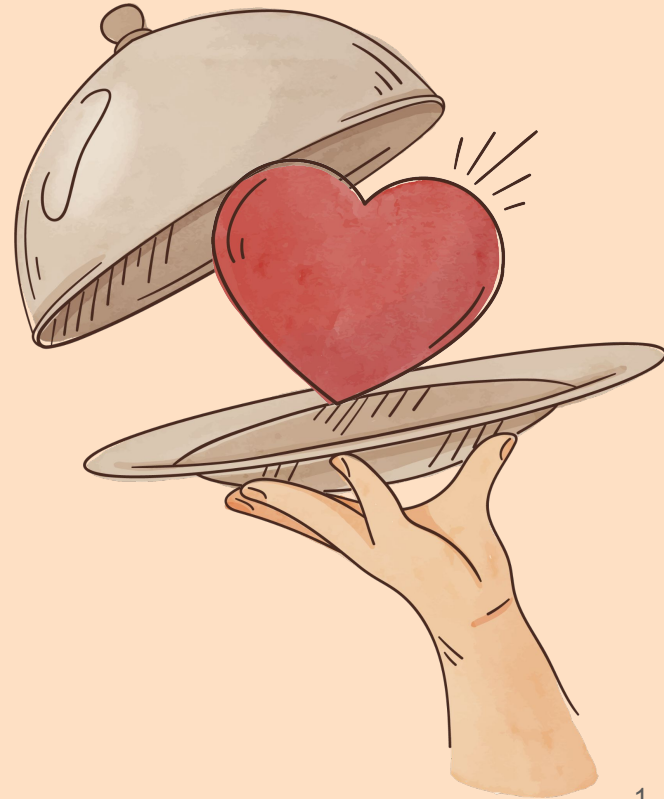


Learning the Vocabulary of Cooking

CS 348K (Spring 2026) - Final Project, Advit Deepak



Recipes are represented as text.

2.2+ million recipes, 195+ cuisines, between 3 and 80 steps per recipe.

Braised Leeks and Lentils with Arugula and Yogurt

SERVINGS: 4 TIME: 1 HOUR SOURCE: SMITTEN KITCHEN

1 bundle leeks (usually 3 to 4)
Olive oil
Kosher salt
Freshly ground black pepper
3 garlic cloves, two thinly sliced, one minced
1 cup dried lentils de puy (200 grams) ([my go-to here](#)), rinsed
1 bundle scallions (usually 4 to 6), thinly sliced (white and green parts)
Finely grated zest and juice of 1 large lemon
3 cups (710 ml) vegetable broth ([my go-to here](#))
1 cup (230 grams) plain, unsweetened yogurt
2 to 3 ounces (55 to 85 grams) baby arugula leaves, or more to taste

Heat your oven: To 350°F (175°C).

Prepare your leeks: Leeks are notoriously sandy. First, lop off any parts of each leek that are darker than light green. Trim the root ends. Peel back layers of your leek until you no longer have visible dirt that cannot be rinsed. Thoroughly rinse off any remaining dirt or grit. Wipe down your counter and knife (lots of dirt and grit there). Then slice the white and pale green parts into 1-inch rings.

serious eats

Save

Step 8

Use tongs and a slotted spoon to remove hoja santa and avocado leaf, if using. Add chocolate and stir until completely melted, about 5 minutes. Mole should be smooth and thick enough to coat the back of a spoon. If too gritty, blend further with an immersion blender, keeping the mole in the pot, or blend in batches in the traditional blender. If too thick, thin with water or chicken stock. Season to taste with salt and sugar. Serve with poached chicken, tortillas, and/or Mexican rice.



Credit: Serious Eats / Lorena Masso

An official website of the United States government [Here's how you know](#)

USDA Nutrition.gov
U.S. DEPARTMENT OF AGRICULTURE

Home About Us Topics Recipes USDA - REE Expert Q&A

[Home](#) > [Recipes](#) > Oatmeal Pecan Waffles

Oatmeal Pecan Waffles

Kids will love these whole-grain waffles made with whole-wheat flour and oats.

Yield

4 servings

Ingredients

1 cup whole wheat flour
 $\frac{1}{2}$ cup oats (quick-cooking)
2 teaspoons baking powder
1 teaspoon sugar
 $\frac{1}{4}$ cup nuts (pecans, unsalted, chopped)
2 eggs (large, separated)
 $1\frac{1}{2}$ cups milk (fat-free (skim))
1 tablespoon vegetable oil
2 cups berries (strawberries, halved)
1 cup berries (blackberries)
1 cup berries (blueberries)
1 teaspoon sugar (powdered)

Steps

1. Preheat waffle iron.
2. Combine flour, oats, baking powder, sugar, and pecans in a large

But, text has several limitations.

Without an LLM, we can't answer any of the following questions:

Substitution

Is this ingredient critical or decorative?

Equipment

How many individual pans are in use at the same time?

Parallelization

What steps can I run simultaneously?

Complexity

What's the hardest step in this recipe?

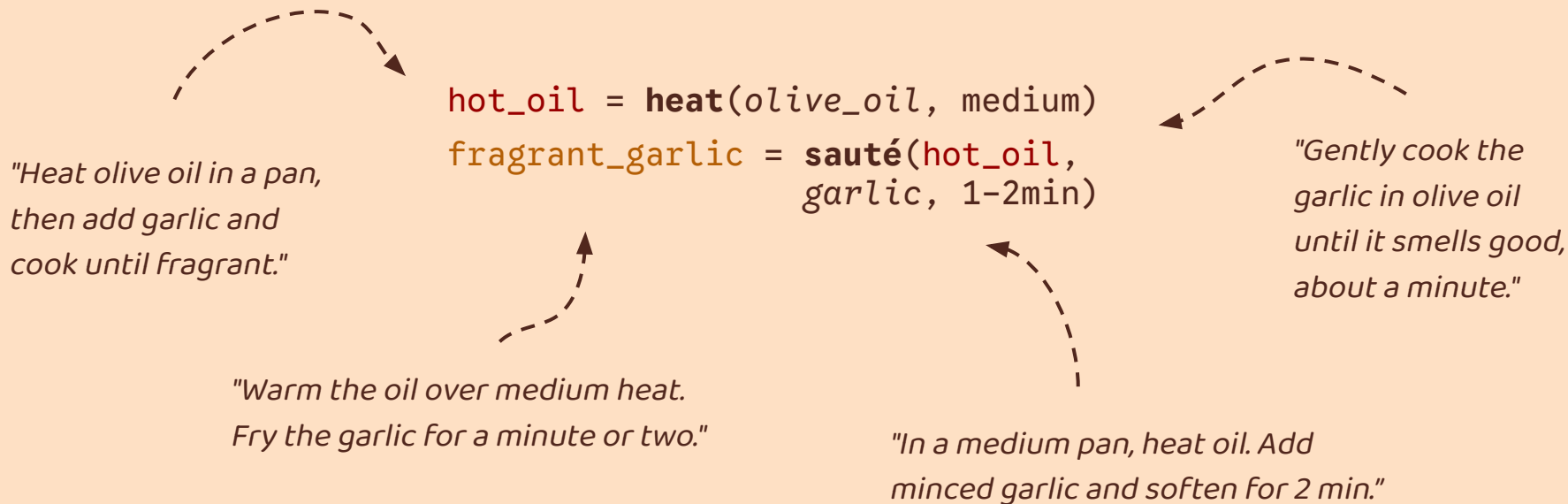
Scaling

If I double the recipe, which steps scale linearly and which don't?

Optimize

How to improve a recipe with respect to some target metric?

Recipe text is a rendering of an underlying recipe program.



So, we think of recipes as programs.

allrecipes.com/recipe/269500/creamy-garlic-pasta/

Ingredients: [2 tsp olive oil, 4 garlic cloves, 2 tbsp butter, 3 cups chicken broth, ½ lb spaghetti, ½ tsp pepper, ¼ tsp salt, 1 cup cheese, ¾ cup heavy cream, 1 ½ tbsp parsley]

1. Heat olive oil in a medium pan over medium heat.
2. Add garlic and stir until fragrant, 1 to 2 minutes. Add butter and stir constantly until melted.
3. Pour in chicken broth; add pepper and salt. Boil.
4. Add spaghetti and cook, stirring occasionally, until tender yet firm to the bite, about 12 minutes. Add more chicken broth if pasta starts to stick to pan.
5. Add Parmesan cheese, cream, and parsley and mix until thoroughly combined. Serve immediately.

```
hot_oil = heat(olive_oil, medium)
fragrant_garlic = sauté(hot_oil,
                        garlic, 1-2min)
garlic_butter = melt(fragrant_garlic,
                    butter)
broth_base = add(garlic_butter, salt,
                chicken_broth, pepper)
boiling_broth = boil(broth_base)
cooked_pasta = cook(boiling_broth,
                   spaghetti, 12min)
final_dish = mix(cooked_pasta,
                 cheese, cream, parsley)
```

Now, we can answer the questions.

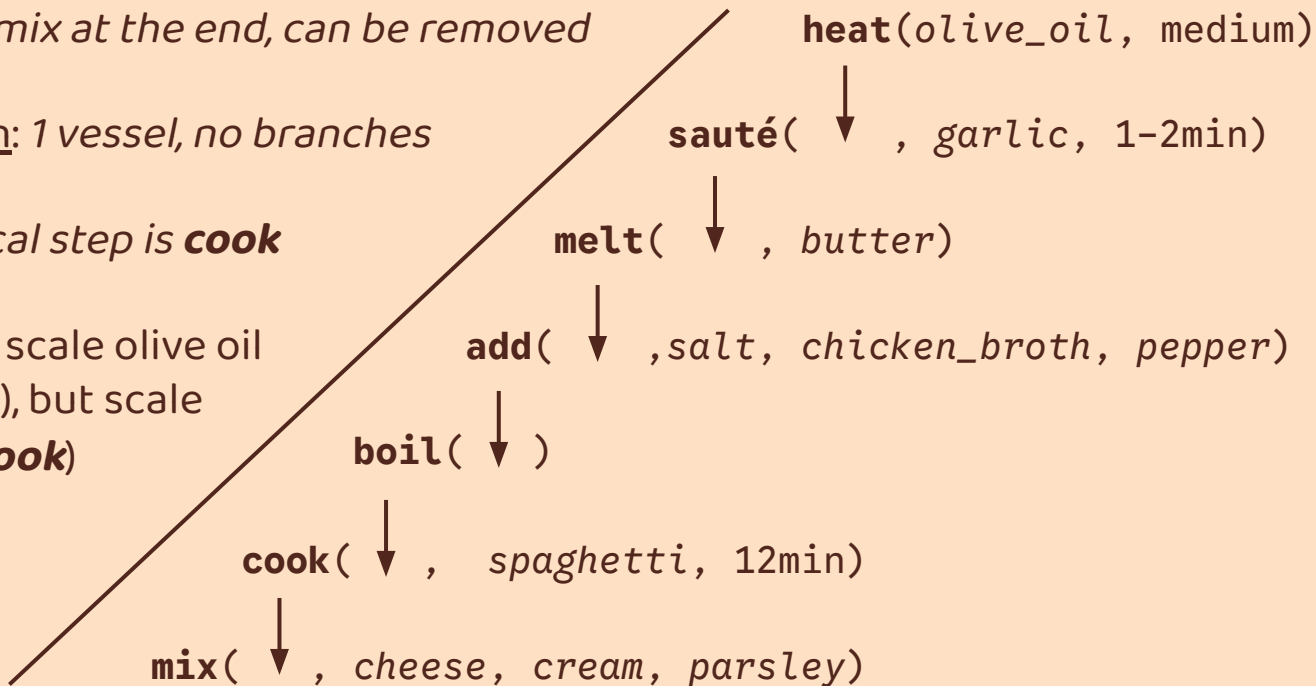
Substitution: *parsley is a mix at the end, can be removed*

Equipment/Parallelization: *1 vessel, no branches*

Complexity: *longest/critical step is **cook***

Scaling (2x): *don't need to scale olive oil (**heat** as transfer medium), but scale broth (*liquid medium in **cook***)*

Optimize: *process graph defines search space*



We need to map steps to processes.

"Sauté onions until translucent, ~5 min"

"Sweat in butter over medium heat"

"Cook leeks until soft and glossy"



[cook] vs. [saute] vs. [soften aromatics in fat]

Are these the same?

"Cook, stirring every few min, until soft and pale golden"

"Reduce heat, stir less often, cook until deep amber"



[stir, wait] vs. [softening, caramelizing]

Physical vs. functional?

"Remove from heat and carefully pour in cream"



If previous step is searing meat → [deglaze]

If previous step is hot roux → [hydrate_starch]

What were the previous steps?

"Cook, stirring constantly"



If onions, then [soften aromatics]. If sugar water then [caramelize]

So, researchers made process DSLs.

Recipe DSL	# Process Primitives	How Vocabulary was Chosen	# Recipes Annotated	Annotation Strategy
Tasse & Smith, 2008	9	Domain Experts	261	Hand-Annotated
Bollini et al., 2013	20	Hand-Crafted for common tasks	60	Hand-Annotated
Mori et al., 2014	15	Linguist-Defined	300	Hand-Annotated
Papadopoulos, 2022	60	Hand-crafted, then refined with iterative K-means	3,708	Amazon Mechanical Turk

But their vocabularies vary widely.

Add() Bake() Cook()
Drain() Drizzle() Mix()
Preheat() Rub() Saute()
Serve() Simmer() Toss()

Subset from Papadopoulos (2022)

add() adjust() bake()
boil() cover() cut()
get() lid() place()
salt() sprinkle() wash()

Subset from Morei et. al (2014)

- Why is cook() separated into bake() but not boil()?
- Why do toss(), mix(), and add() all exist?
- Can drizzle() and sprinkle() be treated the same?

So none of these DSLs are used.

Reason: Arbitrary primitive choice

- Disagreement on terminology:
 - *Ex. 134 terms (Seoul University) vs 173 cooking techniques (Wikipedia)*
- Expert bias when hand-crafted

Reason: Can't scale to 2M+ recipes

- Tried on 128GB VRAM GPU, with Qwen2.5-14B LLM
- Getting ~1600 recipes / hour
- For 2.2M recipes, would take ~57 days → impractical!

A good DSL must have a balanced vocabulary and fast text extraction.

Goal A: Optimal Cooking Vocabulary

- Quantify tradeoff: % of covered recipes with # of vocab terms

Success if: our vocab is semantically coherent and encodes more signal than a human-designed DSL.

Goal B: Recipe Program Generator

- A cheaper, faster way to convert text into useful recipe programs*

Success if: our method produces graphs structurally indistinguishable from a 14B LLM* and be 100x faster.



Goal A

Let's learn the
right vocabulary
of processes.

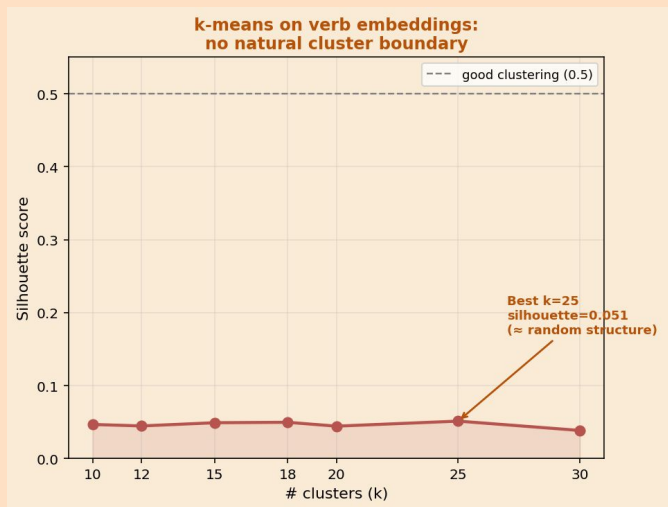
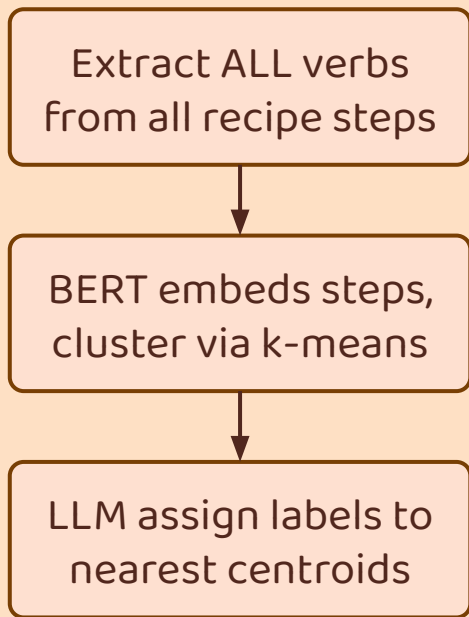
We start with a 1M recipe dataset.

- MIT's Recipe1M+ Data
- 51K / 1.03M recipes are labeled with cuisine and nutritional info

```
"cuisine": "Mexican",  
  
"nutrition": { "calories":  
350, "fatContent": "12.5 g",  
"carbohydrateContent": "25.0  
g", "proteinContent": "28.0  
g", "sodiumContent": "450 mg",  
"sugarContent": "5.0 g" },
```

```
{  
  "id": "000018c8a5",  
  "title": "Spicy Stuffed Peppers",  
  "url":  
"http://www.food.com/recipe/spicy-stuffed-pepper  
s-12345",  
  "partition": "train",  
  "ingredients": [  
    { "text": "3/4 lbs. lean beef" },  
    { "text": "1 cup chopped onion" },  
    { "text": "1/2 cup water" }  
  ],  
  "instructions": [  
    { "text": "Preheat oven to 350 degrees F." },  
    { "text": "Combine all ingredients." },  
    { "text": "Bake for 45 minutes or until  
      peppers are tender." }  
  ]  
}
```

Standard K-Means ignores context.



No natural cluster boundary. Silhouette score (*how similar is this point to its own cluster vs. next-nearest cluster*) ≈ 0

Verb tokens $\neq$ operations $\rightarrow$ context matters

Instead, what if we learn these primitives from the ground-up.

Following GENIE's Vector Quantization (VQ) approach,

Embed each step
of the recipe via
text model

Using MiniLM model
*Ex. Step i: Gently cook
garlic in oil until it
smells good $\rightarrow V_i$*

Causal Encoder to
encode context-
aware current state

*Incorporate self-attn
across previous steps
($V_{1:i}$) and cross-attn on
ingredient embeddings*

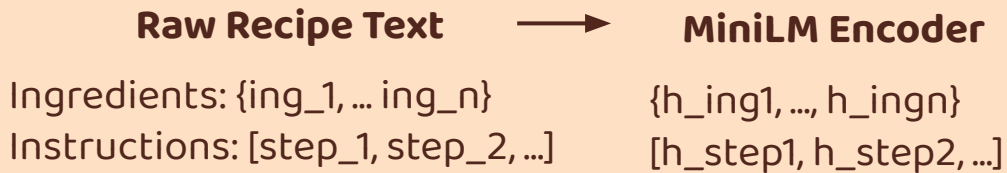
VQ to assign
step to one of K
discrete codes

*Exponential
Moving Average
(EMA) updates to
maintain stability*

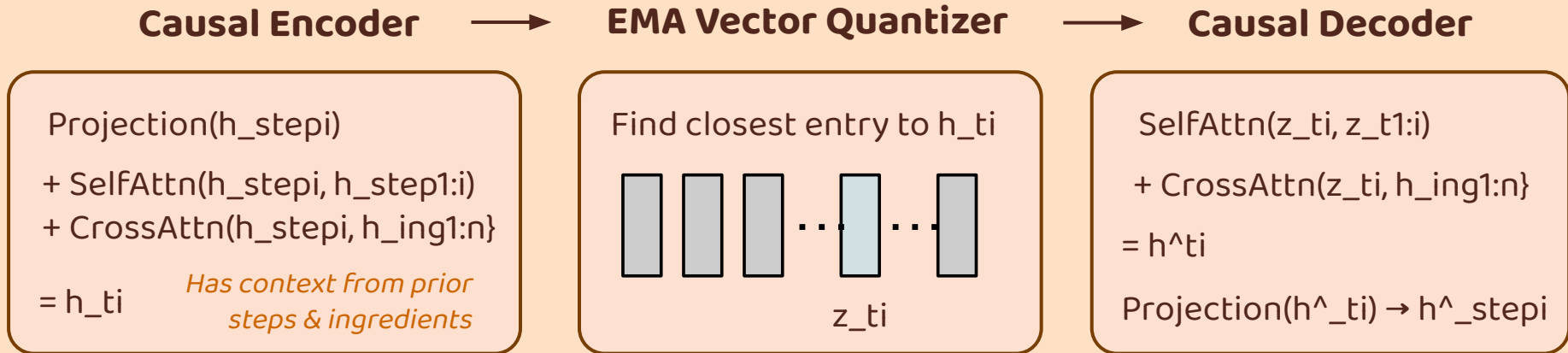
Decoder to
reconstruct
original step
embedding

*Self-attn over
discrete codes
and cross-attn
over ingredients*

Architecture



For a step_i,



Training Objective: Reconstruction Loss (h_{stepi}, h^{stepi}) + VQ Commitment Loss

We see that learned codes (K=256) correspond to meaningful signals.

Assign each code a label by LLM inspection of top-activated step texts.

Code 54

[prepare-pan]

"Dot with 2 teaspoons of the butter.," "Add butter, stirring until melted.," "Add butter 1 piece at a time"

Code 126

[refrigerate]

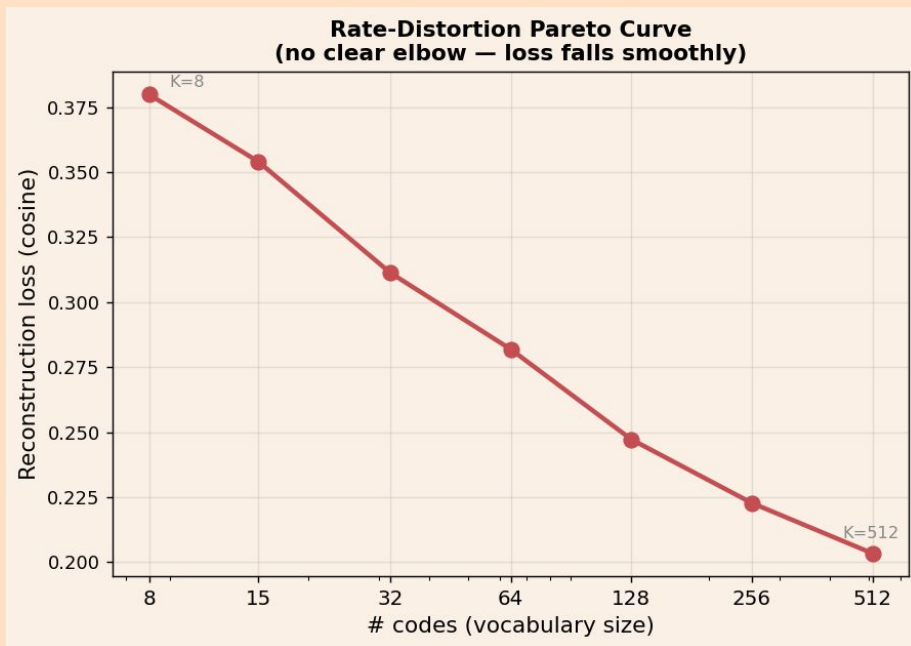
"Cover with lid or foil and refrigerate for about 30 min", "Freeze at least 2 hours", "If time allows, leave in fridge"

Code 153

[garlic saute]

"Add garlic and cook until fragrant, 30 seconds", "In a large saucepan, stir fry ginger and garlic in hot oil 3 min"

Training across varying codebook sizes, there's no clear elbow...



- Log scale (appropriate for vocab), gains are uniform...
- No principled k to stop at
- We use 256 as a practical balance (larger than human expert's upper bound)



Goal B

But, we can use these
learned embeddings
to turn recipe text
into recipe graphs.

Make DAGs from Attention Weights

(1) Run inference on a recipe

(2) Apply Thresholding Rules

Vocab VQ Model: For a step_i,

→ **Causal Decoder**

SelfAttn($z_{t,i}$, $z_{t1:i}$)
+ CrossAttn($z_{t,i}$, $h_{ing1:n}$)
= h^t_i
Projection(h^t_i) → $h^t_{step_i}$

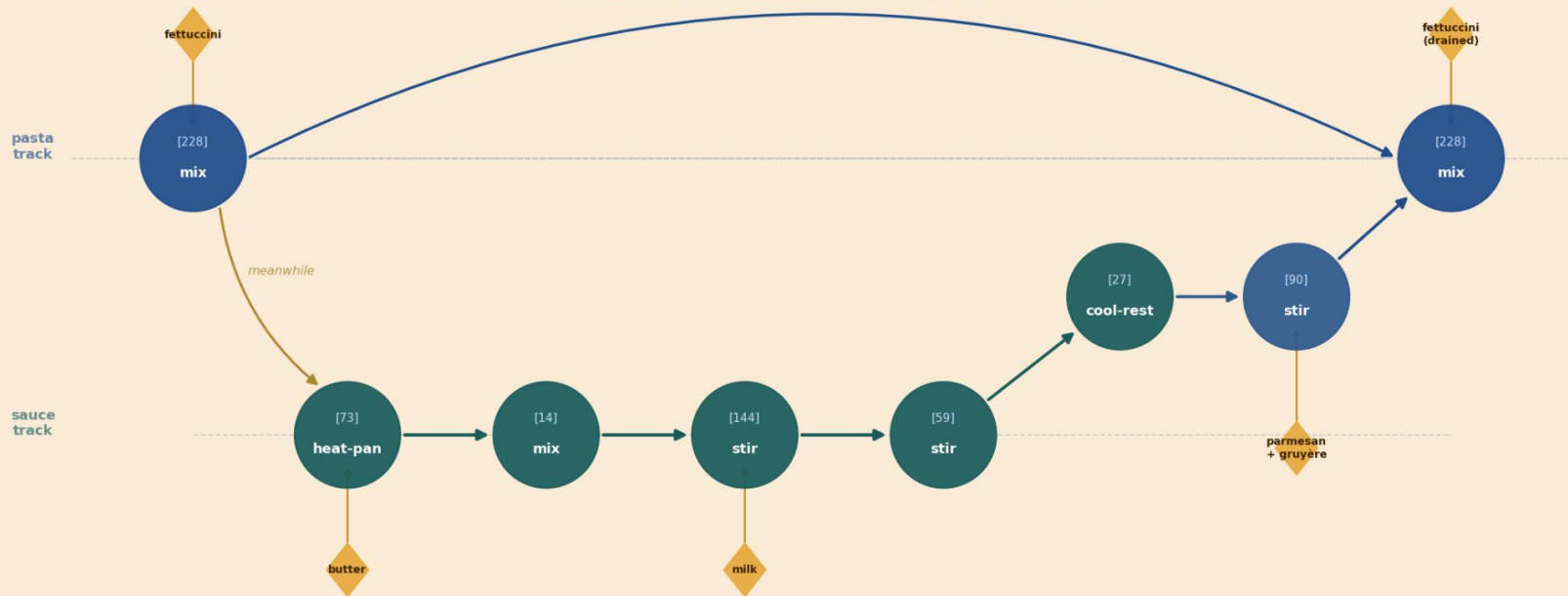
How much step t attends to step s . Add edge for node $s \rightarrow t$ if $\text{SelfAttn}(t, s) > 1/T$ where T is total # of steps

How much step t attends to each ingredient. Add edge for ingredient → node t if $\text{CrossAttn}(t, \text{ing}) > 1/8^*$

(3) Recipe DAG

Example: Fettuccini

Two Cheese Fettuccini — Learned VQ-VAE DAG

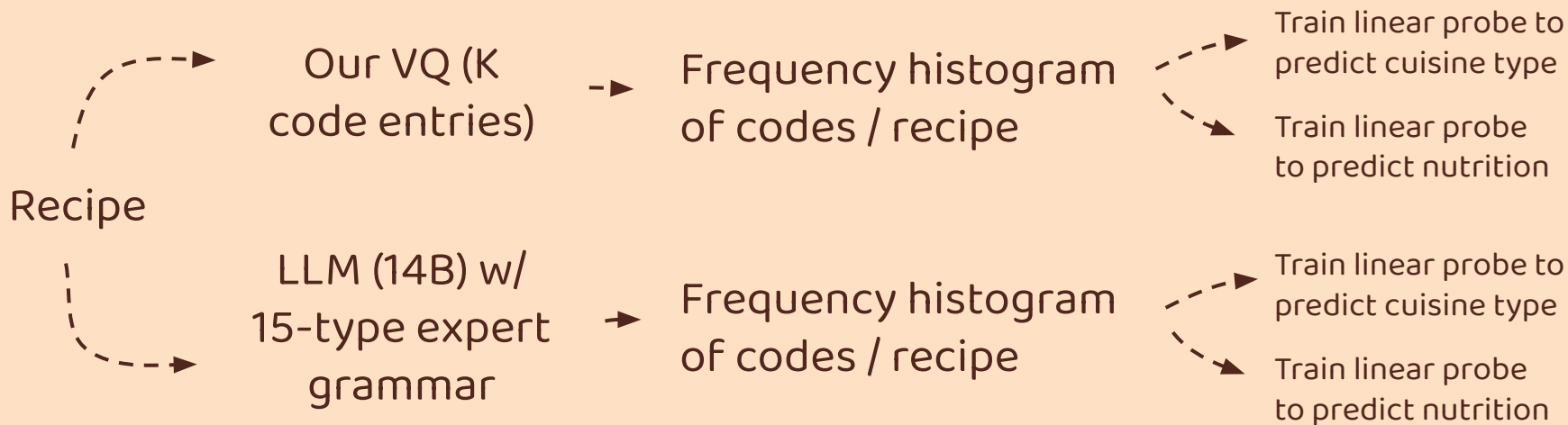


Our method produces DAGs of comparable quality to a 14B LLM.

- Generated 500 graphs from recipes using:
 - Our learned VQ method (non-LLM)
 - Qwen2.5-14B with constrained decoding using Mori et al's 15-primitive grammar
- LLM-as-a-Judge with Claude Sonnet 4.6
- Our win rate: 52.6% → indistinguishable!

And it encodes rich representations.

Recipe1M has ~51K recipes with cuisine and nutrition data.



And it encodes rich representations.

Task	Grammar (15)	VQ K=15	VQ K=32	VQ K=64	VQ K=128	VQ K=256
Cuisine AUC	0.696	0.684	0.697	0.703	0.723	0.728
Nutrition AUC	0.655	0.661	0.699	0.729	0.756	0.772

- The learned vocab scales while the grammar can't, and the win on nutrition indicates that its learning some useful new signal

Our method is also much faster.

Metrics	LLM (14B) using 15 process expert grammar and constrained decoding	Process VQ-VAE (ours)
Recipes Processed	35,867	1,029,518
Total Time	22.2 hours	4.3 minutes
Throughput	~1617 / hour	14.4M / hour
Speedup	-	~8900x

Our steps toward a recipe DSL.

Goal A - Learning the Vocabulary

- No principled K - pareto curve has no elbow
- Model discovers sub-types humans wouldn't write down
- Learned vocab scales monotonically

Goal B - Recipe Program Generator

- A 9MB encoder achieves parity with a 14B LLM on DAG quality
 - Claude Sonnet judge: 52.6% win rate on 500 recipes
- Codes capture richer signal than grammar types (ex. nutrition)
- ~9000x faster → generated DAGs for 1M+ recipes

Where we'll go from here...

Limitations to Overcome

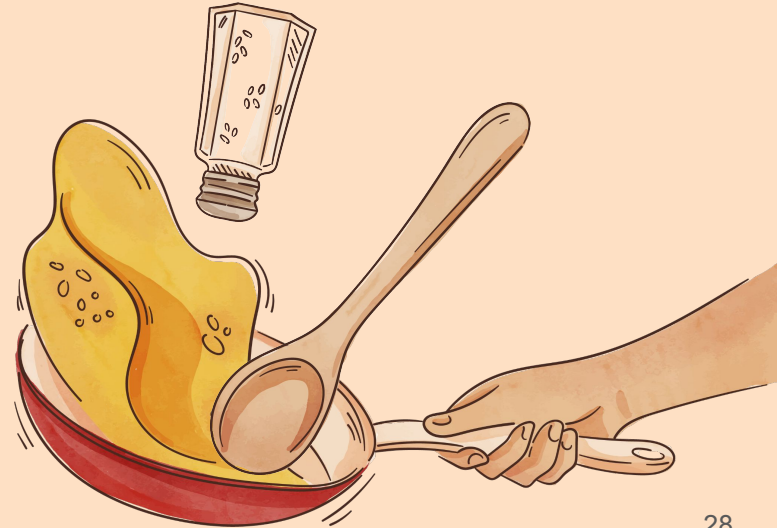
- No ground truth — all comparisons are via LLM-as-judge against small LLMs
- Encoder bottleneck — MiniLM was not trained on recipes

What's on the Horizon

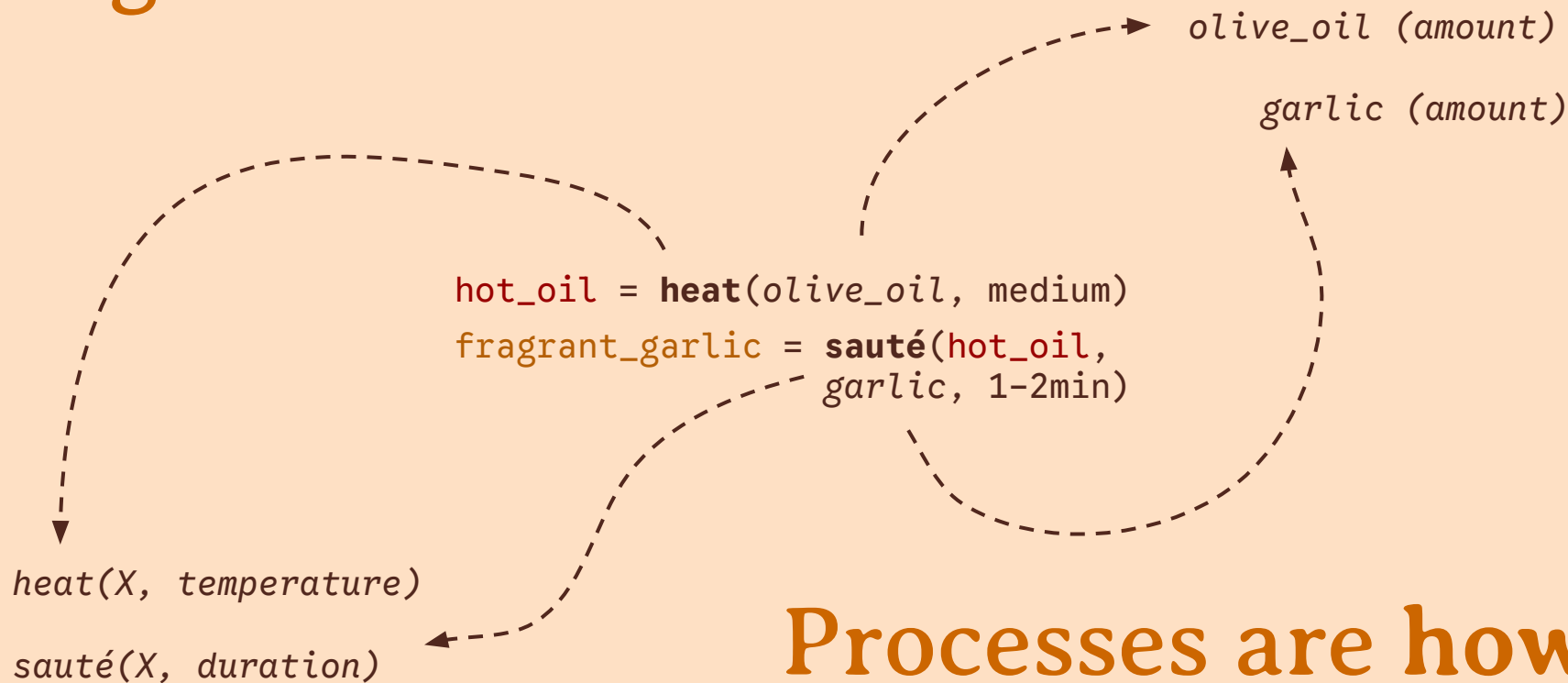
- Can optimize these learned recipes
 - Ex. SRD → fix ingredient set, steer toward target by swapping codes
- Beyond recipes, lots of text descriptions are inherently recipes/DAGs
 - Cocktails/brewing, wet lab protocols, skincare routines

Thanks!

Now serving: questions!



Ingredients are what.



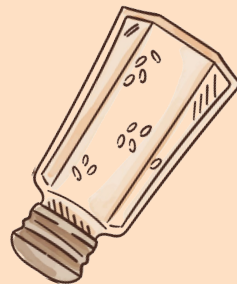
But, there are too many ingredients for each to be a unique primitive.

Thousands of possible ingredients, with many regional variations. We can't assign a unique code to each.

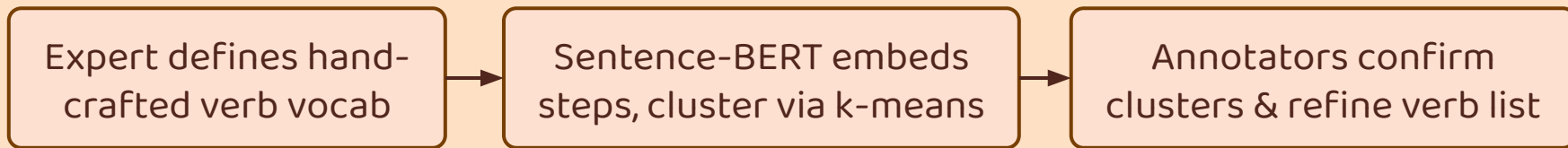
- Recipe1M+ (MIT): ~16,000 unique ingredients
- FoodKG (RPI): ~12,000 canonical ingredients
- USDA Food DB: ~8,000 food items

Napkin Numbers:

- Average supermarket sells ~30,000 food items (FMI).
- [Estimated over 200,000 unique ingredients. \(Ledra Capital\)](#)



Best prior work used hand-crafted primitives refined with k-means.



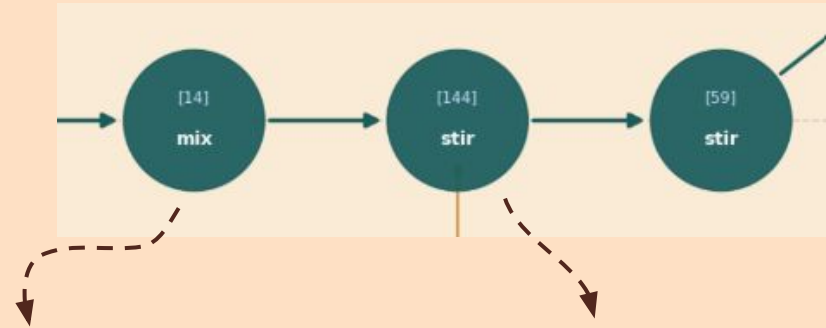
Problems:

- Expert bias baked in – can't escape human prior
- No learned representations of those verbs
- No principled stopping criterion → why end at 60?

Example (cont.)

Two Cheese Fettuccini:

1. Cook fettuccini according to pkg instructions.
2. Meanwhile, over low heat, melt butter.
3. Stir in flour, salt and pepper.
4. Slowly stir in milk; bring to a boil.
5. Cook and stir for 2 minutes, or until thickened.
6. Remove from heat.
7. Toss cheeses with lemon juice; add to milk, stir until melted.
8. Mix with drained fettuccini.



Code 14 [mix]

*"Stir in flour with wooden spoon,"
"Cook over medium heat, stirring until mixture thickens"*

14: Flour-specific, maybe why ingredient wasn't tagged

Code 144 [stir]

"At this stage you could also choose to add goat yoghurt to the mix"; "Slowly pour in milk, while stirring or whisking."; "Stir until well combined, adding more rice milk as needed"

144: Add milk/liquid dairy to mixture

Example (cont.)

LLM couldn't turn sequence of instructions into parallel structure.

Two Cheese Fettuccini — Grammar-15 DAG (14B LLM)

